Submission - E1 Kafka - Test Results

- Course: Event-driven and Process-oriented Architectures (EDPO), FS2026, University of St.Gallen
- Assignment: Exercise 1 - Kafka Getting Started
- Group 4
  - Evan Martino
  - Marco Birchler
  - Roman Babukh

Quick Access Links

- Repository: https://github.com/MrStarco/EDPO_FS26_E1_Kafka_Tests
- Main branch: https://github.com/MrStarco/EDPO_FS26_E1_Kafka_Tests/tree/main
- Marco branch: https://github.com/MrStarco/EDPO_FS26_E1_Kafka_Tests/tree/marco
- Roman branch: https://github.com/MrStarco/EDPO_FS26_E1_Kafka_Tests/tree/roman
- Evan branch: https://github.com/MrStarco/EDPO_FS26_E1_Kafka_Tests/tree/evan

Scope and Method

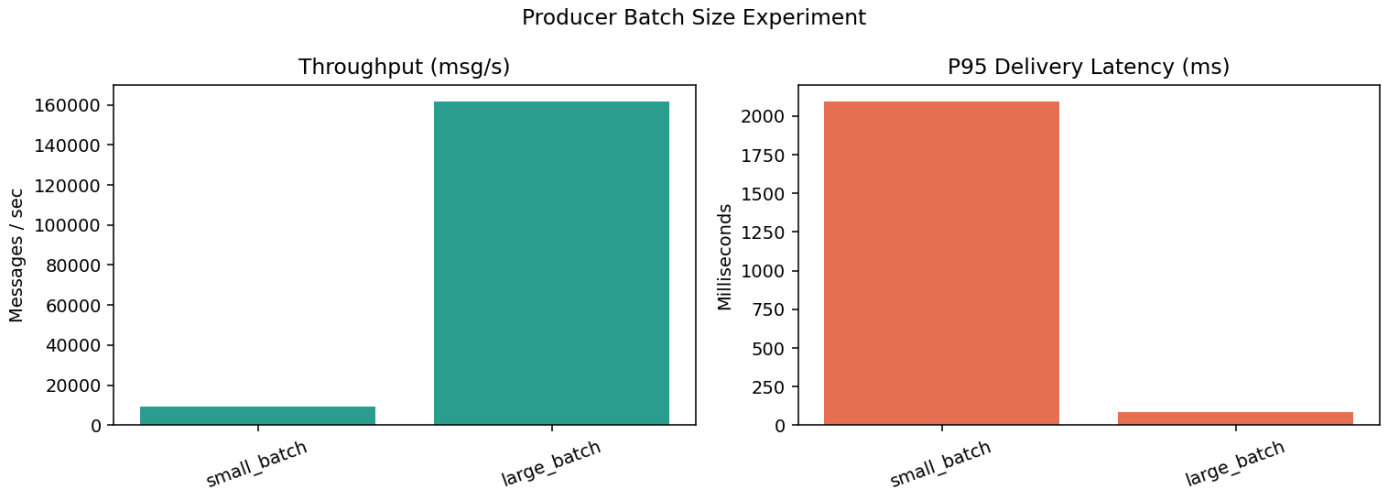
This document is the combined, high-level report. It consolidates results from:

- `marco` branch
- `roman` branch
- `evan` branch

1) Producer Experiments

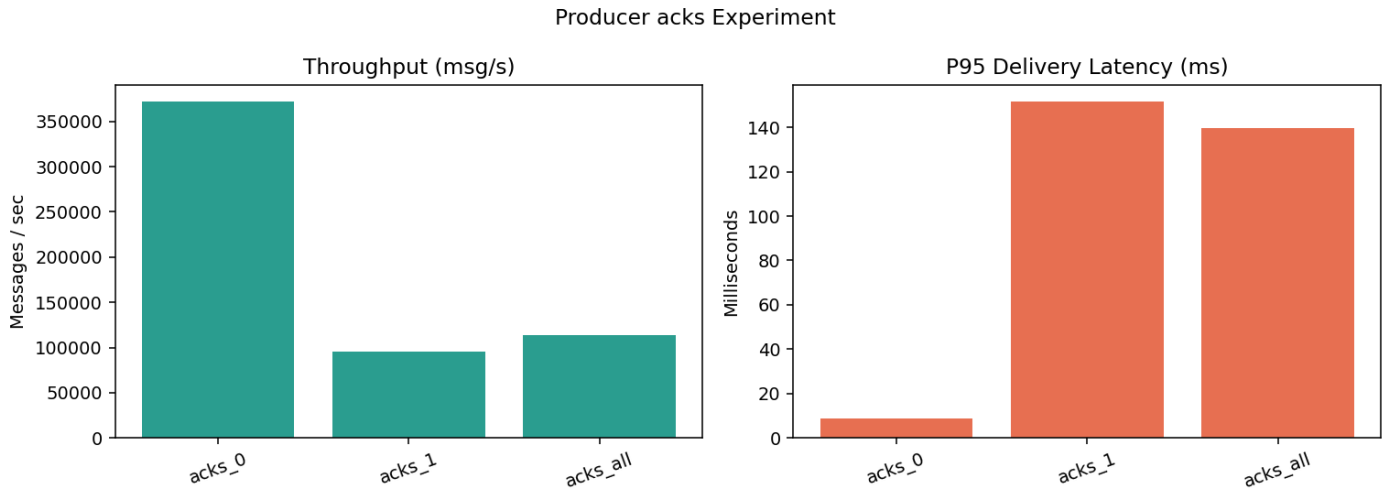
1.1 Batch Size and Processing Latency

- Tested: two producer batching strategies over `20,000` messages, one sending small batches immediately and one collecting much larger batches before sending.
- Result: in the 3-broker setup, the larger-batch strategy performed far better, reaching `161763.76 msg/s` versus `9195.66 msg/s` and lowering p95 latency to `85.241 ms` versus `2095.639 ms`. Neither run produced delivery errors.



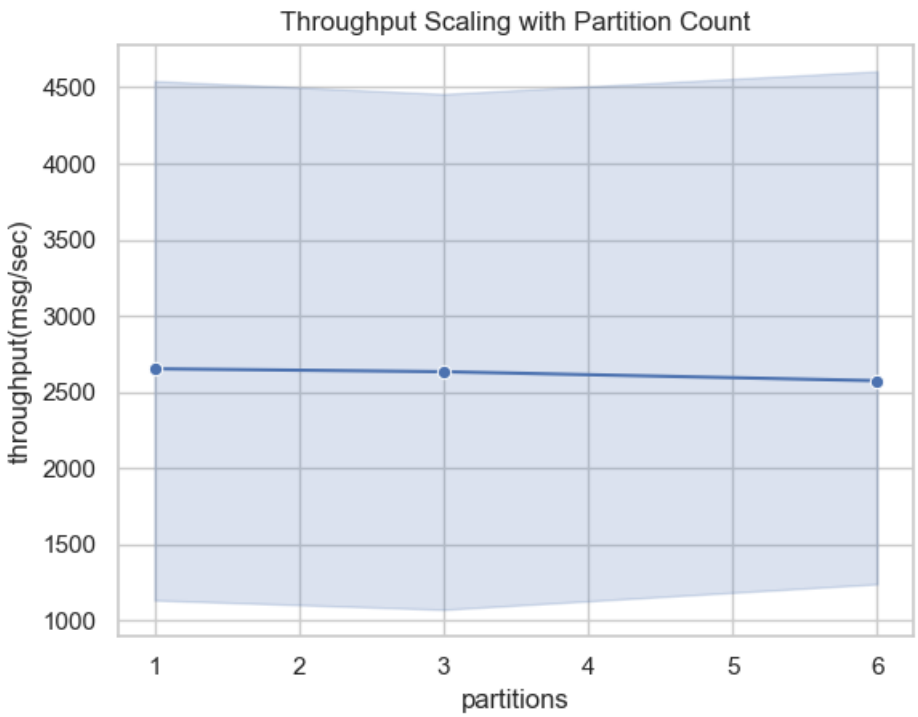
1.2 Acknowledgment Configuration

- Tested: three delivery confirmation levels, from no broker confirmation to full replica confirmation.
- Result: the least durable setting was the fastest in both result sets. In the measured benchmark it reached `371761.9 msg/s` with p95 latency `8.815 ms`, while the two stronger confirmation settings reached `95213.9 msg/s` and `113533.63 msg/s` with p95 latencies of `151.516 ms` and `139.867 ms`. All three runs delivered all `20,000` messages without errors.



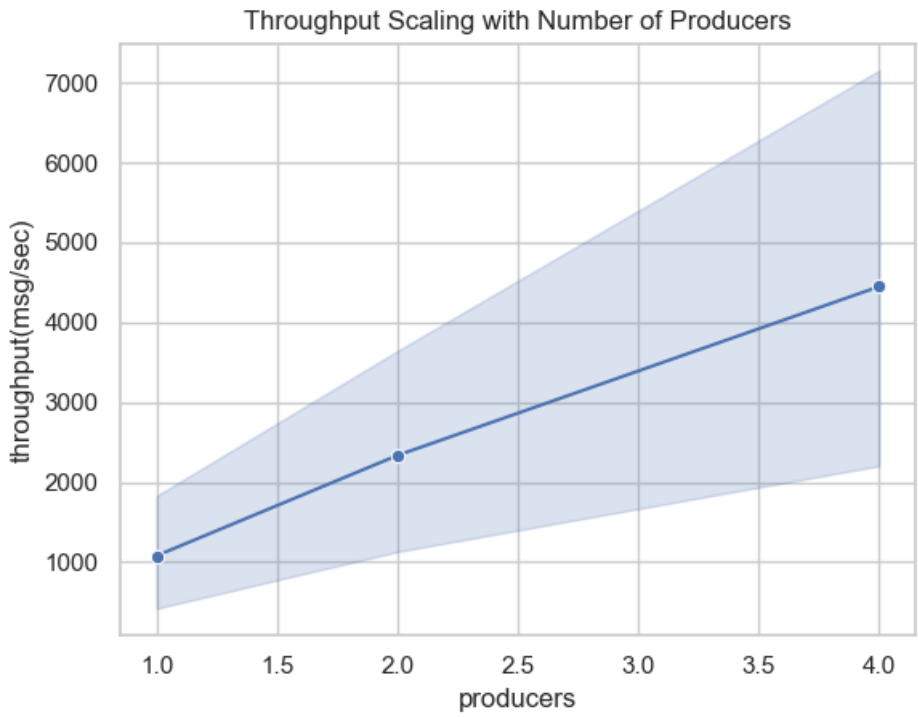
1.3 Brokers and Partitions

- Tested: throughput impact of increasing the number of topic partitions in the single-broker setup.
- Result: adding more partitions did not materially improve throughput. In this setup, all partitions still ran on the same broker and the same laptop hardware, so the bottleneck remained unchanged.



1.4 Load Testing

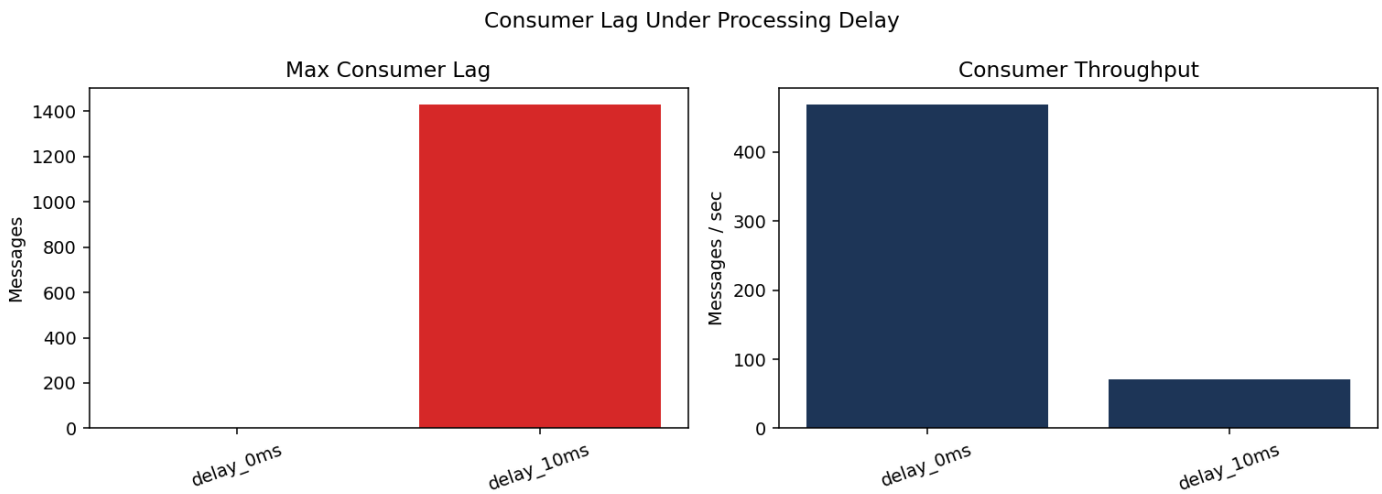
- Tested: throughput scaling as the number of producers increased in parallel.
- Result: throughput increased as more producers ran in parallel. The improvement was smaller when stronger delivery guarantees were enabled, which shows that parallelism helps, but broker confirmation still limits peak throughput.



2) Consumer Experiments

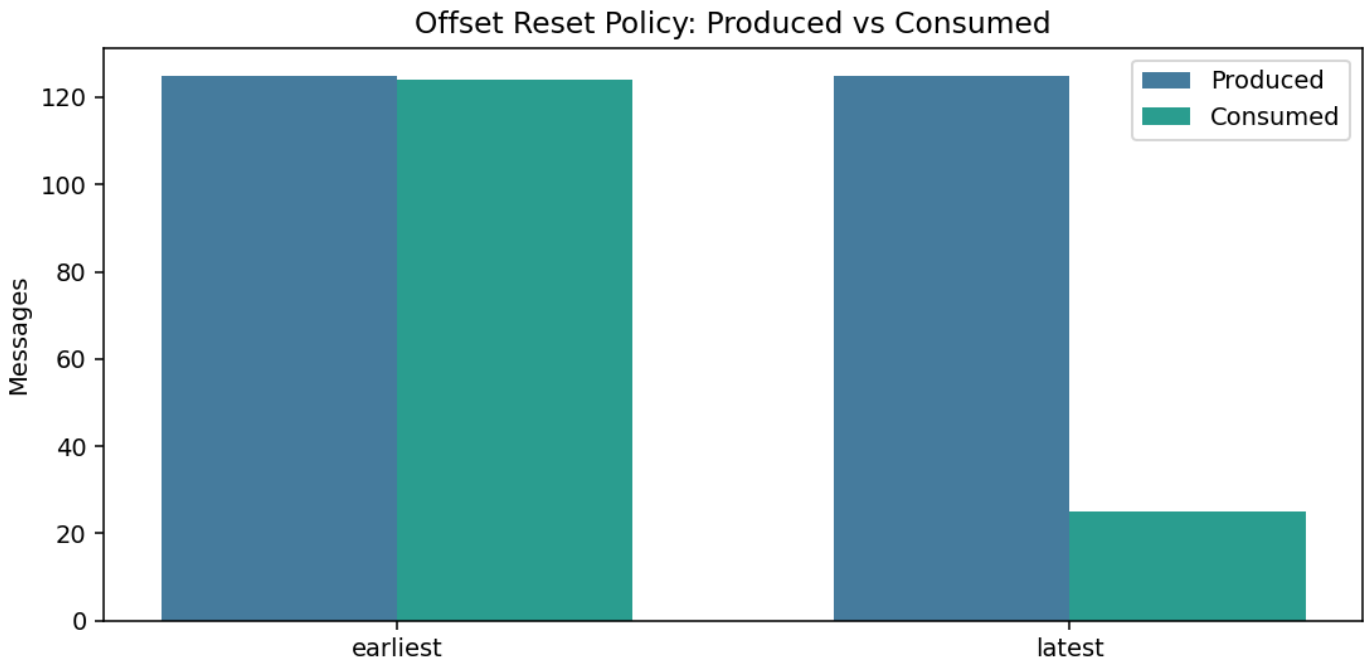
2.1 Consumer Lag Under Processing Delay

- Tested: consumer lag and throughput for `1,500` messages, once with no artificial delay and once with a `10 ms` delay per message.
- Result: the added delay increased maximum lag from `0` to `1431` messages and reduced throughput from `469.32` to `70.83 msg/s`. Both runs still finished all `1,500` messages, but the slower run took `21.179 s` instead of `3.196 s`.

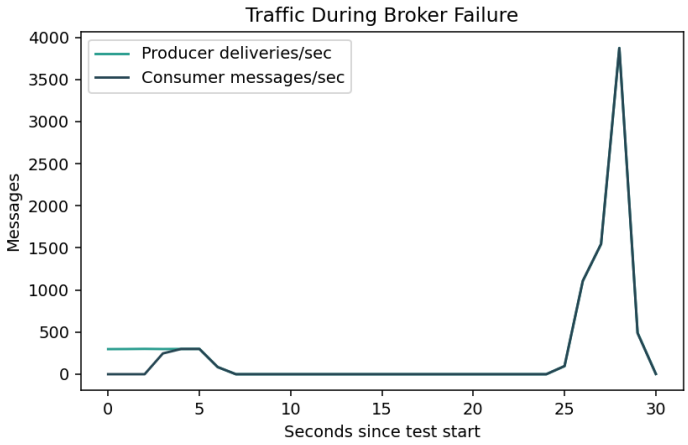
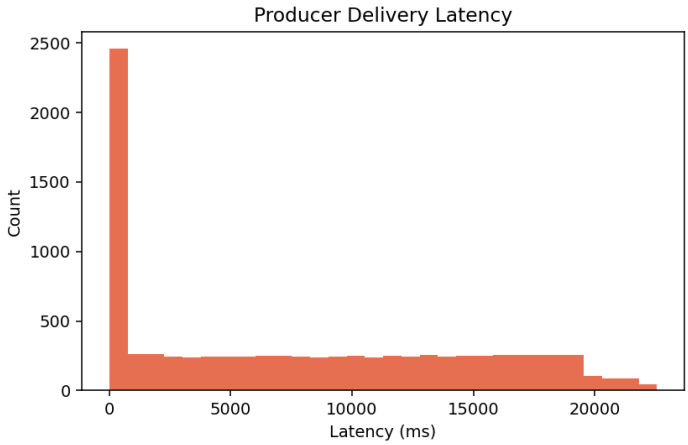


2.2 Offset Misconfiguration

- Tested: a new consumer group joining after `100` messages were already in the topic, once starting from the beginning and once starting only from newly arriving messages. After that, `25` additional messages were produced.
- Result: the consumer that started from the beginning read `99` of the `100` existing messages and all `25` new ones. The consumer that started only from new data read none of the existing messages and all `25` new ones, effectively skipping `100` historical records. This demonstrates the practical data-loss risk of choosing the wrong starting position for a new group.



3) Fault Tolerance and Reliability

3.1 Broker Failure and Leader Election

- Tested: broker failover during a `30 s` run on a topic with `6` partitions and replication factor `3`; broker `3` was killed after a `6 s` warmup.
- Result: leader failover affected `2` partitions and took `19.714 s`. The producer still delivered `9001/9001` messages with `0` errors, while the consumer processed `8049` messages and experienced a maximum gap of `19.682 s`. Producer p95 latency spiked to `19176.295 ms` during failover.